Day - 24

All About Pandas Library for Data Analysis **in Python**

SkillCourse
E-learning Platform by Satish Dhawale

A.I. Powered 30 Days Python Micro Course By Satish Dhawale (Microsoft Certified Trainer)

What is Pandas?

Technical Definition : Pandas is an **open-source Python library** for **data manipulation and analysis**, providing fast, flexible, and expressive **data structures**:

Series → 1-D labeled array

DataFrame → 2-D table-like structure

Built on top of **NumPy**.

Layman Explanation

Pandas is like **Microsoft Excel inside Python**,but **1000x more powerful**, and capable of handling **millions of rows** easily.

Read	Read Excel / CSV files
Clean	Clean data
Transform	Transform data
Analyze	Analyze data
Prepare	Prepare dashboards
Remove	Remove missing values
Fix	Fix wrong spellings
Group	Group data
Filter	Filter data
Export	Export clean data

Who Created Pandas?

Pandas was created in **2008** by:

👤 **Wes McKinney**

A data analyst at a financial company (AQR Capital).

He needed a tool to:

Analyze financial time-series data

Clean messy datasets

Process millions of data rows quickly

None of the existing Python tools worked, so he built:

👉 **Pandas = PANel + Data**

Later:

Pandas became open-source

Developed under **NumFOCUS Foundation**

Now used globally in every data science job

Why Was Pandas Created?

Before Pandas:

- Python had no proper data-table structure
- Only option was NumPy arrays
- Cleaning text, dates, missing data was difficult
- Excel was slow for large datasets
- R language had better data tools → Python had none
- Pandas solved all these limitations.

Why Pandas Is Essential for Data Analysts?

- ✓ **Handle millions of rows**
- ✓ **Load CSV, Excel, SQL in seconds**
- ✓ **Fix messy data quickly**
- ✓ **Remove duplicates, missing values**
- ✓ **Filter and transform easily**
- ✓ **Grouping, aggregation, pivots**
- ✓ **Combine multiple files**

Installing & Importing Pandas

```
pip install pandas
```

```
import pandas as pd
```

Pandas Data Structures

A) Series

A one-dimensional labeled array.

```
s = pd.Series([10, 20, 30])
print(s)
```

B) DataFrame (VERY IMPORTANT)

A DataFrame is like an Excel sheet.

```
data = {
   "Name": ["Rohit", "Sneha"],
   "Age": [25, 30],
   "City": ["Mumbai", "Pune"]
}
df = pd.DataFrame(data)
print(df)
```

Reading Data Files

✓ **Read CSV**

```
df = pd.read_csv("day21_sales_data.csv")
```

✓ **Read Excel**

```
df = pd.read_excel("sample.xlsx")
```

✓ **Read JSON**

```
df = pd.read_json("data.json")
```

DataFrame Basic Operations

✓ **Show first 5 rows**

df.head()

✓ **Show last 5 rows**

df.tail()

✓ **Show shape (rows, columns)**

df.shape

✓ **Show column names**

df.columns

✓ **Summary statistics (very useful)**

df.describe()

Selecting Rows & Columns

✓ **Select one column**

```
df["City"]
```

✓ **Select multiple columns**

```
df[["Customer_Name", "City", "Price"]]
```

✓ **Select rows with condition**

```
df[df["City"] == "Mumbai"]
```

Adding New Columns

✓ **Calculate Total Amount**

```
df["Total"] = df["Quantity"] * df["Price"]
```

✓ **Create new column based on condition**

```
df["Expensive"] = df["Price"] > 20000
```

✓ **Deleting Columns**

```
df = df.drop("Payment_Type", axis=1)
```

Handling Missing Values

✓ **Check missing values**

```
df.isnull().sum()
```

✓ **Replace missing values with average**

```
df["Price"].fillna(df["Price"].mean(), inplace=True)
```

✓ **Drop rows containing missing data**

```
df.dropna(inplace=True)
```

Handling Duplicates

✓ **Detect duplicates**

df.duplicated().sum()

✓ **Remove duplicates**

df.drop_duplicates(inplace=True)

String Cleaning in Pandas

✓ **Trim spaces**

```
df["City"] = df["City"].str.strip()
```

✓ **Title Case**

```
df["Customer_Name"] = df["Customer_Name"].str.title()
```

✓ **Lower Case / Upper Case**

```
df["City"] = df["City"].str.lower()
```

✓ **Replace values**

```
df["City"] = df["City"].replace({"mombai": "Mumbai"})
```

Filtering Rows

✓ **Price > 20000**

```
df[df["Price"] > 20000]
```

✓ **Multiple conditions**

```
df[(df["City"] == "Mumbai") & (df["Price"] > 20000)]
```

Sorting

✓ **Sort by price**

```
df.sort_values("Price")
```

✓ **Sort descending**

```
df.sort_values("Price", ascending=False)
```

GroupBy

✓ **Total Sales per City**

```
df.groupby("City")["Price"].sum()
```

✓ **Average Price per Product**

```
df.groupby("Product")["Price"].mean()
```

✓ **Merging & Joining**

```
merged = pd.merge(df1, df2, on="Customer_ID")
```

✓ **Export Clean Data**

Save cleaned CSV

```
df.to_csv("cleaned_sales.csv", index=False)
```